README_DOCKER

HelixCode Docker Setup

Complete containerized environment for HelixCode with network access, distributed processing, and easy management.

Quick Start

```
# 1. Make the facade script executable
chmod +x helix

# 2. Setup environment
cp .env.example .env
# Edit .env with your preferences

# 3. Start everything (or let commands auto-start)
./helix start

# 4. Access services
./helix status  # View connection info
./helix tui     # Open Terminal UI (auto-starts if needed)
./helix cli --help  # Use CLI (auto-starts if needed)
```

Prerequisites

- Docker and Docker Compose
- 4GB+ RAM recommended
- Git (for repository access)

Architecture

Container Stack

- **helixcode**: Main application with REST API, CLI, and Terminal UI
- **postgres**: PostgreSQL database for persistent storage
- **redis**: Redis cache for sessions and queues
- **worker-1/worker-2**: Distributed worker nodes

Network Configuration

- **Internal Network**: `helixcode-network` (172.20.0.0/16)
- **Port Mapping**: Configurable via environment variables
- **Service Discovery**: Automatic in distributed mode

Configuration

Environment Variables (.env)

Run `./setup.sh` to have these generated for you: it writes `.env` at mode 0600 with a unique random secret per install, and never overwrites a value you have already set. The compose files and the server hold no credential of their own (CONST-042) — they read these variables and refuse to start if one is missing.

```
# Security (REQUIRED) — generated by ./setup.sh, or set a unique
        random value
# per install by hand (e.g. `openssl rand -hex 24`). Never reuse
        a published value.
HELIX_DATABASE_PASSWORD=your-secure-password
HELIX_AUTH_JWT_SECRET=your-jwt-secret
HELIX_REDIS_PASSWORD=your-redis-password

# Port Configuration
HELIX_API_PORT=8080      # REST API port
HELIX_SSH_PORT=2222      # Worker SSH port
HELIX_WEB_PORT=3000      # Web interface port

# Network Mode
HELIX_NETWORK_MODE=standalone  # standalone or distributed
HELIX_AUTO_PORT=true           # Auto-adjust if ports occupied
        (recommended)
```

Directory Structure

```
helix_code/
├── helix                    # Main facade script
├── Dockerfile               # Main application image
├── docker-compose.helix.yml # Complete stack definition
├── docker-entrypoint.sh     # Container entrypoint
├── .env.example            # Environment template
├── DOCKER_SETUP.md         # Detailed documentation
├── test-docker-setup.sh    # Comprehensive test suite
└── README_DOCKER.md        # This file
```

Usage

Using the Facade Script

```
# Start services (or let commands auto-start)
./helix start

# Check status and connection info
./helix status

# Run CLI commands (AUTO-STARTS if needed)
./helix cli --list-workers
./helix cli --health
./helix cli --prompt "Hello world" --model llama-3-8b

# Open Terminal UI (AUTO-STARTS if needed)
./helix tui

# View logs
./helix logs
./helix logs helixcode

# Stop services
./helix stop

# Restart
./helix restart
```

Auto-Start Feature

Commands like `cli`, `tui`, and `server` automatically start the container if it's not running:

```
# No need to run 'start' first - it happens automatically!
./helix cli --help          # Auto-starts container
./helix tui                 # Auto-starts container
./helix cli --list-models   # Auto-starts container
```

Direct Container Access

You can also access the container directly:

```
# Execute commands in container
docker exec -it helixcode helix cli --help
```

```
# Access shell
docker exec -it helixcode /bin/bash

# View running processes
docker exec helixcode ps aux

# Note: Use ./helix commands for auto-start convenience
```

Network Access

Service URLs (After Startup)

```
Available Services:
• REST API:       http://localhost:8080
• SSH Workers:    localhost:2222
• Web Interface:  http://localhost:3000

Accessible Directories:
• Workspace:      ./workspace
• Projects:       ./projects
• Shared:         ./shared
```

Network Modes

Standalone Mode (Default)

- Single container with all services
- Simple setup, ideal for development

Distributed Mode

- Multiple containers can connect
- Automatic service discovery
- Enable with: `HELIX_NETWORK_MODE=distributed`

Testing

Run Complete Test Suite

```
./test-docker-setup.sh
```

Manual Testing

```
# Test auto-start feature
./helix cli --health        # Should auto-start container
./helix tui                 # Should auto-start container

# Test service connectivity
curl http://localhost:8080/health

# Test CLI functionality
./helix cli --list-workers
./helix cli --list-models

# Test worker connectivity
docker exec helixcode-worker-1 echo "test"

# Cleanup
./helix stop
```

Troubleshooting

Common Issues

Port Conflicts:

```
# Check what's using ports
netstat -tulpn | grep :8080

# Or use auto-port adjustment (enabled by default)
HELIX_AUTO_PORT=true ./helix start
```

Container Not Starting:

```
# Check logs
./helix logs

# Check Docker daemon
docker info

# Check resources
docker system df
```

Worker Connection Issues:

```
# Check SSH setup
ls -la helix_code/test/ssh-keys/

# Test worker connectivity
docker exec helixcode ssh worker-1 echo "test"
```

Resource Requirements

- **Minimum**: 2GB RAM, 2 CPU cores
- **Recommended**: 4GB RAM, 4 CPU cores
- **Production**: 8GB+ RAM, 8+ CPU cores

Security

Best Practices

1. **Change default passwords** in `.env`
2. **Use HTTPS** in production
3. **Restrict network access** to necessary ports
4. **Regular updates** of container images
5. **Resource limits** for containers

Environment Security

- `.env` file is gitignored
- Sensitive tokens stored securely
- Database passwords encrypted
- JWT secrets randomized

Production Deployment

Optimized Configuration

```
# Increase resources in docker-compose.helix.yml
deploy:
  resources:
    limits:
      cpus: '4.0'
      memory: 8G
```

External Services

```
# Use external databases
HELIX_DATABASE_URL=postgres://user:pass@host:port/db
HELIX_REDIS_URL=redis://user:pass@host:port/db
```

Monitoring

```
# Container stats
docker stats

# Log monitoring
./helix logs -f

# Health checks
curl http://localhost:8080/health
```

Documentation

- DOCKER_SETUP.md - Detailed setup guide
- HelixCode README - Main project documentation
- API Documentation - REST API reference

Support

For issues: 1. Check `./helix logs` 2. Verify `./helix status` 3. Run `./test-docker-setup.sh` 4. Review troubleshooting section

License

Part of the HelixCode project. See main project LICENSE for details.